

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5 - REKURSIF



DEVIA DIAN INDRIYANTI

109082500150

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Rekursif adalah cara dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah. Tujuannya adalah memecah masalah yang besar menjadi bagian yang lebih kecil sampai menemukan kondisi paling sederhana yang disebut base case. Base case ini penting karena menjadi titik berhenti agar program tidak berjalan terus tanpa henti. Selain itu, ada juga bagian recursive case, yaitu saat fungsi terus memanggil dirinya sendiri dengan nilai yang makin mendekati kondisi dasar.

Cara kerja rekursif bisa dibayangkan seperti turun tangga lalu naik lagi. Saat turun, fungsi terus memanggil dirinya sendiri sampai mencapai base case. Setelah itu, hasilnya akan dihitung saat “naik” Kembali. Karena setiap pemanggilan fungsi disimpan di memori, kalau tidak hati-hati bisa membuat program error. Jadi, yang penting dalam rekursif adalah selalu punya base case dan memastikan setiap Langkah makin dekat ke kondisi tersebut.

Rekursif sering dipakai untuk contoh sederhana seperti factorial atau deret Fibonacci. Walaupun terlihat membingungkan di awal, sebenarnya konsepnya cukup sederhana jika dipahami pelan-pelan: pecah masalah, selesaikan yang paling kecil, lalu gabungkan hasilnya.

B. Guided

1. Modul 5 – Guided 1

```
package main
import "fmt"
func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Guided\Guided1\lat1.go"
5
5
4
3
2
1
PS C:\109082500150_Devia-Dian-Indriyanti> 
```

Penjelasan :

Program ini digunakan untuk menampilkan deretan angka dari suatu nilai input hingga ke angka 1 secara menurun menggunakan metode rekursif. Program dimulai dengan membaca sebuah angka dari pengguna, kemudian memanggil fungsi baris untuk memproses dan mencetak angka tersebut. Ketika nilai yang diproses sudah mencapai 1, program akan berhenti, sehingga tidak terjadi pemanggilan fungsi secara terus-menerus.

Setiap nilai akan dicetak terlebih dahulu sebelum fungsi memanggil dirinya kembali dengan nilai yang lebih kecil, yaitu dikurangi 1. Hal ini membuat output yang dihasilkan berupa urutan angka menurun dari angka awal hingga 1.

2. Modul 5 – Guided 2

```
package main
import "fmt"
func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

```
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Guided\Guided2\lat2.go"
5
15
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Guided\Guided2\lat2.go"
10
55
PS C:\109082500150_Devia-Dian-Indriyanti> |
```

Penjelasan :

Program ini digunakan untuk menghitung jumlah angkadari 1 sampai nilai yang diinput oleh pengguna dengan menggunakan rekursif. Pada fungsi main, program membaca input n, lalu memanggil fungsi penjumlahan(n) dan menampilkan hasilnya. Fungsi ini memiliki kondisi berhenti saat $n == 1$, Dimana nilai 1 langsung dikembalikan.

Jika n lebih dari 1, fungsi akan mengembalikan hasil $n + \text{penjumlahan}(n-1)$, sehingga proses penjumlahan dilakukan secara bertahap hingga mencapai 1. Misalnya jika input 5, maka hasilnya adalah $5 + 4 + 3 + 2 + 1 = 15$.

3. Modul 5 – Guided 3

```
package main
import "fmt"
func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return 2 * pangkat(n-1)
    }
}
```

```
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Guided\Guided3\lat3.go"
4
16
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Guided\Guided3\lat3.go"
5
32
PS C:\109082500150_Devia-Dian-Indriyanti> 
```

Penjelasan :

Program ini digunakan untuk menghitung nilai perpangkatan 2^n menggunakan metode rekursif. Pada fungsi main, program membaca input angka n, kemudian memanggil fungsi pangkat(n) dan menampilkan hasilnya. Fungsi pangkat memiliki kondisi berhenti saat $n == 0$, dimana hasilnya adalah 1 karena 2^0 bernilai 1.

Jika nilai n lebih dari 0, maka fungsi akan mengembalikan hasil $2 * \text{pangkat}(n-1)$. Proses ini akan terus berulang hingga mencapai 0, sehingga secara bertahap menghitung $2 \times 2 \times 2$ dan seterusnya sebanyak n kali.

C. Unguided

1. Modul 5 – Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret Fibonacci tersebut.

Jawaban :

```
package main
import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    n := 10
    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Unguided\Unguided1\fibonacci.go"
0 1 1 2 3 5 8 13 21 34 55
PS C:\109082500150_Devia-Dian-Indriyanti>
```

Penjelasan :

Program ini digunakan untuk menampilkan deret Fibonacci menggunakan metode rekursif. Fungsi Fibonacci(n) memiliki dua kondisi dasar yaitu saat $n == 0$ mengembalikan 0 dan $n == 1$ mengembalikan 1. Jika nilai n lebih dari 1, maka fungsi akan mengembalikan hasil dari penjumlahan dua nilai sebelumnya, yaitu $\text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$.

Pada fungsi main, program menetapkan nilai $n = 10$, lalu menggunakan perulangan untuk mencetak deret Fibonacci dari indeks 0 sampai 10. Setiap

angka dihitung menggunakan fungsi rekursif, sehingga output yang dihasilkan adalah deret seperti 0 1 1 2 3 5 8 13 21 34 55.

2. Modul 5 – Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Jawaban :

```
package main
import "fmt"

func faktorRekursif(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    faktorRekursif(n, i+1)
}

func main() {
    var n int
    fmt.Print("masukkan N: ")
    fmt.Scan(&n)
    fmt.Print("faktor: ")
    faktorRekursif(n, 1)
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 5-Modul 5\Unguided\Unguided2\faktorBilangan.go"

masukkan N: 5
faktor: 1 5
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devia-Dian-Indriyanti\Week 5-Modul 5\Unguided\Unguided2\faktorBilangan.go"
masukkan N: 12
faktor: 1 2 3 4 6 12
PS C:\109082500150_Devia-Dian-Indriyanti> █
```

Penjelasan :

Program ini digunakan untuk menampilkan semua factor dari suatu bilangan yang diinput oleh pengguna dengan menggunakan metode rekursif. Pada fungsi main, program meminta input angka, lalu memanggil fungsi faktorRekursif(n, 1) untuk mulai mengecek factor dari angka 1 sampai n. Fungsi ini akan berhenti ketika nilai i sudah lebih besar dari n.

Didalam fungsi faktorRekursif, setiap nilai i akan dicek apakah merupakan factor dari n dengan kondisi $n \% i == 0$. Jika iya, maka nilai i akan dicetak. Setelah itu, fungsi akan memanggil dirinya sendiri dengan nilai i+1 untuk melanjutkan pengecekan berikutnya. Dengan cara ini, program akan menampilkan semua factor dari bilangan tersebut secara berurutan.

3. Modul 5 – Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Jawaban :

```
package main
import "fmt"

func tampilGanjil(i, n int) {
    if i > n {
        return
    }
    fmt.Print(i, " ")
    tampilGanjil(i+2, n)
}

func main() {
    var n int
    fmt.Print("masukkan N: ")
    fmt.Scan(&n)
    tampilGanjil(1, n)
}
```


Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Unguided\Unguided3\bilanganGanjil.go"

masukkan N: 5
1 3 5
PS C:\109082500150_Devia-Dian-Indriyanti> go run "c:\109082500150_Devi
a-Dian-Indriyanti\Week 5-Modul 5\Unguided\Unguided3\bilanganGanjil.go"
masukkan N: 20
1 3 5 7 9 11 13 15 17 19
PS C:\109082500150_Devia-Dian-Indriyanti> █
```

Penjelasan :

Program ini digunakan untuk menampilkan semua factor dari suatu bilangan yang diinput oleh pengguna dengan menggunakan metode rekursif. Pada fungsi main, program meminta input angka n, lalu memanggil fungsi faktorRekursif(n, 1) untuk mulai mengecek factor dari angka 1 sampai n. Fungsi ini akan berhenti ketika nilai i sudah lebih besar dari n.

Didalam fungsi faktorRekursif, setiap nilai i akan di cek apakah merupakan factor dari n dengan kondisi $n \% i == 0$. Jika iya, maka nilai i akan dicetak. Setelah itu, fungsi akan memanggil dirinya sendiri dengan nilai i+1 untuk melanjutkan pengecekan berikutnya. Dengan cara ini, program akan menampilkan semua factor dari bilangan tersebut secara berurutan.

D. Kesimpulan

Kesimpulan dari materi rekursif adalah rekursif merupakan metode pemrograman dimana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah secara bertahap. Dalam setiap rekursif selalu ada kondisi berhenti (*base case*) agar proses tidak berjalan tanpa akhir, serta bagian proses yang terus mengulang dengan nilai yang semakin mendekati kondisi tersebut.

Rekursif biasanya digunakan pada masalah yang memiliki pola berulang, seperti penjumlahan deret, faktorial, Fibonacci, perpangkatan, dan pencarian faktor bilangan. Dengan cara ini, masalah yang terlihat besar bisa dipecah menjadi bagian yang lebih kecil dan diselesaikan satu per satu sampai selesai.

Namun, penggunaan rekursif juga perlu diperhatikan karena jika tidak dirancang dengan baik bisa menyebabkan proses yang lambat atau penggunaan memori yang besar. Oleh karena itu, penting untuk memahami alur rekursif dengan benar agar dapat digunakan secara efektif dan efisien dalam pemrograman.